UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Closure

São funções anônimas que podem ser salvas em variáveis ou passadas como parâmetro para funções. Ao contrário de funções *closures* podem capturar o contexto onde foram criados.

```
fn  inc_v1   (x: u32) -> u32 { x + 1 }

let inc_v2 = |x: u32| -> u32 { x + 1 };
let inc_v3 = |x|             { x + 1 };
let inc_v4 = |x|               x + 1  ;

let a = 10;
let foo = |x| x + a;
```



Ponteiro para Funções

```
fn map (f:fn(&i32) -> T2, v1:&Vec<i32>) -> Vec<i32>{
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}
fn main() {
    let v1 = vec![10, 20, 30];
    let v2;

    v2 = map(|x| x + 1, &v1);
    println!("{v2:?}");
}
```

UDESC

Ponteiro para Funções

```
fn map<T1, T2> (f:fn(&T1) -> T2, v1:&Vec<T1>) -> Vec<T2>{
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}
fn main() {
    let v1 = vec![10, 20, 30];
    let v2;
    let v3 = vec!["programacao", "em", "rust"];
    let v4;

    v2 = map(|x| x + 1, &v1);
    println!("{v2:?}");
    v4 = map(|s| s.to_uppercase(), &v3);
    println!("{v4:?}");
}
```



Closure

```
fn twice<F>(f:F, x:i32) -> i32 where
    F: Fn (i32) -> i32 {
        f(f(x))
    }

fn inc (x:i32) -> i32
{
   x + 1
}

fn main() {
  let a = 10;
  println!("{}", twice (inc, 5));
  println!("{}", twice (|x| x + a, 5));
}
```



Closure

```
fn map<F, T1, T2> (f:F, v1:&Vec<T1>) -> Vec<T2> where
   F : Fn(&T1) -> T2 {
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}

fn main() {
    let a = 10;
    let v1 = vec![10, 20, 30];
    let v2:Vec<i32>;

    v2 = map(|x| x + a, &v1);
    println!("{v2:?}");
}
```



Closure (mutável)

```
fn mapMut<F, T1, T2> (mut f:F, v1:&Vec<T1>) -> Vec<T2> where
   F : FnMut(&T1) -> T2 {
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}
fn main() {
    let mut a = 10;
    let v1 = vec![10, 20, 30];
    let v2:Vec<i32>;
    v2 = mapMut(|x| {a += 1; x + a}, &v1);
    println!("{v2:?}");

  }
```



Iterator

```
fn main() {
  let mut a = 1;
  let v1 = vec![10, 20, 30];
  let v:Vec<i32> = v1.iter().map(|x| x + 1).collect();
  println!("{v:?}");
  let vm:Vec<i32> = v1.iter().map(|x| {a *= 2; x + a}).collect();
  println!("{vm:?}");
}
```

UDESC

Iterator

Iterator é um padrão de projeto que permite percorrer uma coleção de objetos sem a necessidade de expor a representação da coleção. Em Rust o comportamento de um iterator é definido pela trait:

```
pub trait Iterator {
 type Item;
 // Required method
 fn next(&mut self) -> Option<Self::Item>;
 …
}
```

UDESC

Iterator

```
struct Fib {
    atual: u128,
    prox: u128
}

impl Fib {
    pub fn new () -> Self {
        Self {atual:0, prox:1}
    }
}
```

UDESC

Iterator

```
pub trait Iterator {
 type Item;
 // Required method
 fn next(&mut self) -> Option<Self::Item>;
 …
}
impl Iterator for Fib {
    type Item = u128;
    fn next(&mut self) -> Option<u128> {
        let aux = self.atual;
        self.atual = self.prox;
        self.prox  = aux + self.atual;
        Some(aux)
    }
}
```

UDESC

Iterator

```
fn main() {
    let mut f1 = Fib::new();
    let mut f2 = Fib::new();

       for _i in 1..10 {
        println!("{}", f1.next().unwrap());
    }
    println!("{}", f2.nth(8).unwrap());
    let f3 = Fib::new().take_while(|x| x < &1000u128);
    for i in f3 {
        println!("{i}");
    }
    let v:Vec<u128> = Fib::new()
                      .take_while(|x| x < &1000u128).collect();
    println!("{v:?}");
}
```

UDESC

for (iterator)

```
for x in lista {
    println!("{}", x);
}
```



```
let mut iter = lista.into_iter();
loop {
    match iter.next() {
        Some(x) => println!("{}", x),
        None => break,
    }
}
```

UDESC

Iterando sobre Listas

```
use std::mem::replace;

#[derive (Debug)]
struct NoLista {
    elem : i32,
    prox: Option<Box<NoLista>>
}

#[derive (Debug)]
struct Lista {
    cab : Option<Box<NoLista>>
}
```

UDESC

Iterando sobre Listas

```
impl Lista {
  fn new() -> Self {
      Lista {cab: None}
  }
  fn insert (&mut self, v:i32) {
     let n = Box::new(NoLista{elem:v,
                        prox:replace(&mut self.cab, None)});
     self.cab = Some (n);
  }
  fn iter(&self) -> IterListaRef<'_> {
        IterListaRef::new(&self.cab)
  }

}
```

UDESC

Iterando sobre Listas (consumindo a lista)

```
struct IterLista {
    prox: Option<Box<NoLista>>,
}
impl Iterator for IterLista {
    type Item = i32;
    fn next(&mut self) -> Option<Self::Item> {
        match replace(&mut self.prox, None) { //consome a lista
            None => None,
            Some(p) => {
                self.prox = p.prox;
                Some(p.elem)
            }
        }
    }
}
```

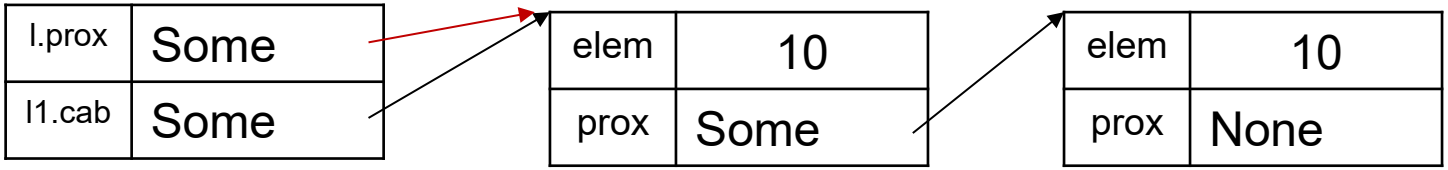


Iterando sobre Listas (consumindo a lista)

```
impl IntoIterator for Lista {
    type Item = i32;
    type IntoIter = IterLista;
    fn into_iter(self) -> Self::IntoIter {
        IterLista {
            prox: self.cab,
        }
    }
}
fn main() {
    let mut l1:Lista = Lista::new();
    l1.insert(1); l1.insert(2);
    for i in l1 { //chama implicitamente into_iter() para Lista
        println!("{i}");
    }
}
```

Iterando sobre Listas (consumindo a lista)

```
fn main()
{
   let mut l1:Lista = Lista::new();
   println!("{l1:?}");
   l1.insert(10);
   l1.insert(2);
   let l = l1.into_iter();
   println!("{l1:?}"); //Erro: l1 não possui mais a propriedade
}                        // que foi transferida para l.
```



Iterando sobre Listas

```
struct IterListaRef<'a> {
    prox: &'a Option<Box<NoLista>>
}

impl<'a> IterListaRef<'a> {
    fn new(c:&'a Option<Box<NoLista>>) -> Self {
        IterListaRef {prox:c}
    }
}
```

'a (lifetime) - o tempo de vida do iterador não pode exceder o tempo de vida da lista (referência que ele armazena).

UDESC

Iterando sobre Listas

```
impl Iterator for IterListaRef<'_> {
  type Item = i32;
  fn next (&mut self) -> Option<i32> {
     match self.prox {
        None => None,
        Some (p) => {
            let e = p.elem;
            self.prox = &p.prox;
            Some(e)
         }
     }
  }
}
```



Iterando sobre Listas

```
impl<'a> IntoIterator for &'a Lista {
    type Item = i32;
    type IntoIter = IterListaRef<'a>;


    fn into_iter(self) -> Self::IntoIter {
        self.iter()
    }
}
```

UDESC

Iterando sobre Listas

```
fn main() {
    let mut l1:Lista = Lista::new();

    println!("{l1:?}");
    l1.insert(4);
    l1.insert(3);
    l1.insert(2);
    l1.insert(1);
    for i in &l1 { //chama implicitamente into_iter() para &Lista
        println!("{i}");  // (&l1).into_iter()
    }
    let v1:Vec<i32> = l1.iter().map(|x|x*4).collect();
    println!("{v1:?}");
}
```